MindForge AI Risk Management Handbook — implementing on GTL/ABG and odd_sdlc

Type: Strategy commentary **Author:** Claude (Sonnet 4.6 review session) **Date:** 2026-05-04 **Subject:** Project MindForge AI Risk Management Handbook (Jan 2026, MAS-led financial-services consortium) **Source artifacts (downloaded local copies; canonical PDFs at `mas.gov.sg/schemes-and-initiatives/project-mindforge`):**

- `~/Downloads/MindForge AI Risk Management Executive Handbook.pdf` (26 pages, Nov 2025)
- `~/Downloads/MindForge AI Risk Management Operationalisation Handbook.pdf` (173 pages, Jan 2026)
- `~/Downloads/MindForge AI Risk Management Implementation Examples.pdf` (23 pages, Jan 2026, four FI case studies)

This is commentary, not law. It maps the MindForge control surface onto GTL/ABG primitives and the odd_sdlc lifecycle, names where the substrate already implements a control, names where new carriers are required, and walks one Consideration end-to-end as a worked example.

What MindForge actually is

The Handbook is a financial-services-specific AI risk management framework written by a consortium of 24 banks, insurers, and capital-markets firms led by MAS, supported by Accenture and three hyperscaler technology partners. It is an industry handbook, not yet supervisory expectation; it is intended to "accompany and support" the proposed MAS Guidelines on AI Risk Management currently in consultation (Nov 2025–Jan 2026), which will become supervisory expectation when finalised in 2026.

The Handbook frames itself as building on FEAT (2018) and Veritas (2020-2023). The continuity is real but the scope is wider: FEAT is principles, Veritas is methodology for FEAT compliance at the model level, MindForge is enterprise-level governance covering traditional AI, generative AI, and agentic AI across the use-case lifecycle.

The structural unit is the **Consideration**. There are 17 Considerations grouped into four Sections:

- **Section 1 — Scope and Oversight** (Considerations 1–2): operating model and governance documents
- **Section 2 — AI Risk Management** (Considerations 3–6): enterprise risk, third-party AI risk, use-case-level risk, AI inventory
- **Section 3 — AI Lifecycle Management** (Considerations 7–15): per-use-case lifecycle from context/design through monitoring/change

- **Section 4 — Enablers** (Considerations 16–17): skills/knowledge/culture and infrastructure

Each Consideration decomposes into Practices (typically 1–7 per Consideration), and each Practice in the Operationalisation Handbook expands into multi-paragraph Operationalisation Guidelines plus cross-references to leading frameworks (NIST AI RMF, ISO 42001, EU AI Act, etc.) in Appendix C.

The framework's core operational claim is **proportionality**: governance scales with risk materiality, and risk materiality is assessed both inherently (before controls) and residually (after controls). Use cases are tiered (DBS calls them risk-materiality tiers approved by the RDU Committee; Prudential uses Immaterial → Group Critical with the AIRAQ questionnaire; Julius Baer pre-approves foundation models so per-use-case assessment can skip duplicated review).

Mapping to the GTL/ABG substrate

The MindForge control surface decomposes into four substrate-level concerns. Each maps onto specific GTL/ABG primitives. Where the substrate already realises the control, the implementation is a configuration choice. Where it does not, a new typed carrier is required.

Governance documents and operating model (Considerations 1–2)

MindForge requires policies, procedures, and standards that institutionalise AI governance, plus named Board and Senior Management responsibilities. The Implementation Examples show this realised as: AIWG / RDU Committee / RAIC / AI Council, plus an enterprise AI policy, plus an AI risk framework document, plus role definitions in HR systems.

GTL/ABG side:

- **Method documents** (`SPEC_METHOD`, `TICKET_METHOD`, `DESIGN_MODULE_METHOD`, `ODD_METHOD` — the STDO surface) realise "policies and standards" at the constitutional layer
- **`Role`** (`gtl.work_model`) realises named capability classes; specific committee-role bindings (e.g., `role://rdu-committee/voting-member`) are tenant-local
- **`Job`** over a published `GraphFunction` realises "Senior Management approves this AI use case before deployment" as a typed semantic work contract, not a process diagram
- **F_H regime** (per CLAUDE.md Bootloader §5) realises human approval as a binding evaluator regime; rule 3 ("human approval does not override deterministic failure") is the right safety rule here — committee approval cannot rescue a use case that fails deterministic checks
- **`ODD_METHOD.md` §11.7 (Manual Walkthrough Rule)** realises the constitutional requirement that automation preserve a walk a human could follow; this is the structural answer to MindForge's accountability requirement

What is missing: MindForge's "horizon-scanning" practice (Consideration 1, Practice 4) — periodic review of the operating model itself — has no direct GTL/ABG carrier. It would be a recurring `Job` over a `RefinementBoundary` whose terminal vector is "operating model still fit for purpose" with F_H attestation. A small addition.

Enterprise risk and third-party AI risk (Considerations 3–4)

MindForge requires AI-specific KRIs, AI-specific enterprise risk taxonomy, third-party AI disclosure, vendor onboarding controls, contract clauses for AI change notification, and AI-specific procurement assessment.

GTL/ABG side:

- **Event stream** (`REQ-R-ABG3-EVENTS`) realises KRI tracking as event-sourced fact: each AI invocation emits structured events, projection derives KRI values by replay
- **Payload-ledger projection** (`REQ-R-ABG3-PAYLOAD`) realises evidence admission for audit; KRI thresholds become payload-ledger query predicates
- **`AgentTransportContract`** with `sanitizedEnvironmentPolicy` realises third-party AI transport boundary — every external agent invocation is a typed contract with declared environment hygiene
- **Traced call-out substrate** (T-108/T-109) realises per-call forensic evidence: when an external AI provider returns unexpected output, the trace archive is the disclosure obligation
- **`AgentTransportFailureClass`** (`transport_failure | no_output | contract_failure`) realises the supervisory distinction between "vendor failed" and "use case rejected the input"

What is missing: MindForge's "AI risk taxonomy" (Appendix B of the Operationalisation Handbook lists seven risk dimensions) is a typed surface that would benefit from being a first-class GTL carrier rather than a domain configuration. A `RiskDimension` carrier with `inherent_severity`, `residual_severity`, and `control_refs` fields would make risk-based dispatch a first-class scheduling concern.

Use-case-level risk management and AI inventory (Considerations 5–6)

MindForge requires risk materiality tiering, per-use-case inherent and residual risk assessment, controls applied proportionate to risk, pre- and post-deployment review, and an AI inventory recording purpose, scope, AI types, data, third-party model info, status, and governance.

GTL/ABG side:

- **`Module`** as publication boundary realises "approved use case registered in the AI inventory"
- **`GraphFunction` × `Job` binding** realises "use case has a named purpose, scope, and ownership"
- **`CandidateFamily`** realises proportionate-control alternatives — the risk tiering chooses among lawful structural alternatives rather than running one process with conditional branches
- **`RefinementBoundary`** realises pre-deployment review: terminal vector is "review attests use case in scope and within risk appetite"
- **Resolved-runtime contract** (`workspace://.ai-workspace/runtime/resolved-runtime.json`) realises a runtime AI inventory; cross-workspace binding (T-104) realises the inventory's authoritative scope

What is missing: explicit risk-tier metadata at the `Module` and `Job` level. MindForge's risk-materiality assessment is structurally a typed annotation on the work contract; no current carrier captures this. Recommended addition: `riskMaterialityTier: "low" | "moderate" |`

`"high" | "group_critical"` on `JobInit` and `ModuleInit`, with a derivation rule from declared characteristics (autonomy level, data sensitivity, blast radius) so the tier is replay-derivable rather than declared by the operator.

AI lifecycle management (Considerations 7–15) — the odd_sdlc surface

This is where odd_sdlc earns its keep. The MindForge lifecycle (use case context & design → data acquisition & processing → onboarding/build/review → deployment → usage/monitoring/change) maps directly onto the odd_sdlc lifecycle (request → specification → design → implementation → qualification → release → deployment → runtime return → observation → retrofit) with one caveat: MindForge collapses the post-deployment phases more tightly than odd_sdlc's "runtime return → observation → retrofit" decomposition.

Mapping:

MindForge Section 3	**odd_sdlc stage**	**GTL/ABG primitive**
C7 use case context & design	request → specification → design	`Job` admission, `Module` declaration, design-module surface
C8–C9 data acquisition & processing	specification (data section) → implementation (data prep edge)	AssetSurface contract, payload-ledger admission, F_D data-validity evaluator
C10 onboarding (third-party)	specification (third-party section) → qualification	`AgentTransportContract`, third-party `Module` import
C11 build	implementation	`GraphFunction` realisation per edge, F_P worker dispatch
C12 pre-deployment review	qualification	F_D structural review at terminal vectors, F_H gate at obligation closure
C13 deployment	release → deployment	T-082 output instance allocation, cross-workspace binding
C14 monitoring	runtime return → observation	event stream, KRI projection, `actor_process_*` events, traced call-out archives
C15 change management	retrofit	ABG correction shadows, reset/reopen with fresh attempt identity, payload-ledger amendment

The Implementation Examples confirm this fit. Prudential's PRUShield Chatbot walks request → AI Registry registration (use case design) → LSRC review (architecture design) → GSRC review (group-level design) → in-house build with RAG → 4-month UAT (qualification) → CAB approval (deployment gate) → phased rollout (deployment) → continuous monitoring + annual recertification (runtime return + retrofit). Each named gate corresponds to a terminal vector in an odd_sdlc graph function. The AIRAQ questionnaire is an F_D evaluator over typed AI-specific attributes.

DBS's CodeBuddy walks the same lifecycle through ALAN (the AI inventory) plus the RDU Committee gate. The "stateless processing" requirement and "secured proxy service for governed interaction" are concrete instantiations of `AgentTransportContract` with sanitized environment policy.

Julius Baer's pattern — pre-approval of foundation models so per-use-case review skips repeated foundation-model assessment — maps onto a published `RefinementBoundary` whose foundation-model attestation is a frame-local convergence already discharged at the publication boundary, leaving per-use-case review to discharge only the use-case-specific obligations.

The Investment Firm's three-body governance ecosystem (AI Council, AI Governance Workgroup, AI Engineering Workgroup) is three named `Role` definitions with three corresponding `Job` contracts, dispatched by `CandidateFamily` selection on the AI workload taxonomy (SaaS / COTS / in-house).

Enablers (Considerations 16–17)

C16 (skills/knowledge/culture) is outside the substrate proper. The closest GTL/ABG hook is the `WRITING_GUIDE.md` and `POSTING_GUIDE.md` companions — they govern how method artefacts are written and how commentary is recorded — plus the explicit Builder/Custodian/Use-Case-Owner role taxonomy.

C17 (infrastructure) maps onto the traced call-out substrate, executor profiles (`local-spawn` / `pty-terminal`), and the trace archive shape published in `build_tenants/common/traced_process/README.md`. The substrate already covers monitoring, scalability, availability, and security at the AI-invocation boundary; non-AI infrastructure controls (cloud security, data centre BCM, etc.) remain non-AI-specific concerns and explicitly out of MindForge's scope per the Executive Handbook §1.1.

Where MindForge requires additions to GTL/ABG

Honest accounting of gaps. None are large; all are typed-carrier additions or small spec drops.

1. **`RiskMaterialityTier` carrier** — first-class risk tier on `Job` and `Module` with replay-derivable assignment from declared use-case characteristics. Currently inferable from policy hooks but not a typed substrate field.
2. **`AIInventoryEntry` carrier or projection** — MindForge requires a specific set of inventory attributes (purpose, scope, AI types, data, third-party model info, status, governance). The resolved-runtime contract is close but does not declare these attributes as required. A projection over `Module` admissions with the MindForge attribute set is one realisation; making it a carrier is another.
3. **Multi-level escalation surface** — MindForge requires escalation from peer review to SME panel to committee to C-suite. F_H is a single regime. The right realisation is `CandidateFamily` of escalation paths gated by inherent risk tier, with each lawful alternative naming the role(s) authorised to admit.
4. **KRI projection helpers** — payload-ledger projection has all the underlying data; canonical KRI projections (latency, error rate, drift, bias, hallucination rate per use case) are not yet shipped. These are not substrate work but reference projections odd_sdlc could publish.
5. **Recertification cadence as graph-function** — MindForge requires periodic recertification with cadence proportionate to risk tier. The right realisation is a `RecurrenceProfile` on `Module` whose terminal vector reopens the post-deployment review job at the declared cadence. ABG's recursive runtime contract supports this; no syntactic sugar exists yet.

6. **Risk taxonomy as typed surface** — MindForge Appendix B lists seven risk dimensions. Making these a typed carrier (rather than tenant-local configuration) means dispatch and selection can route on them.

One Consideration walked end-to-end

Concrete worked example, **Consideration 5 — use-case-level AI risk management**, on GTL/ABG + odd_sdlc.

MindForge requires (paraphrasing the six Practices): 1. Define risk materiality levels for AI use cases 2. Assess inherent risk materiality at lifecycle stage 3. Assess residual risk materiality before deployment 4. Apply controls commensurate with risk 5. AI-specific review prior to deployment 6. AI-specific reviews periodically post-deployment

Realisation:

```
Module:        odd_sdlc/use_case/<id>
RiskTier:      derived from CandidateFamily over (autonomy, data_sensitivity, blast_radius)
Job:           job://<id>/risk-managed-deployment
GraphFunction: gf://<id>/use-case-deployment-with-tier-<tier>-controls
Edges:
  e1 inherent_risk_assessment -> InherentRiskTier (F_D evaluator over typed attributes)
  e2 control_selection         -> CandidateFamily(control-set-low | control-set-mod | control-
set-high)
  e3 build                     -> implementation edge per chosen control set
  e4 residual_risk_assessment  -> ResidualRiskTier (F_D over admitted controls)
  e5 ai_specific_review        -> F_H attestation (committee gate; role binding by tier)
  e6 deploy                    -> T-082 output instance allocation
  e7 monitor                   -> recurring graph function, cadence by tier
  e8 recertify                 -> reopen e5 at declared cadence; correction shadow on changes
```

Lifecycle stage mapping (odd_sdlc): e1 in design; e2 in design; e3 in implementation; e4 in qualification; e5 in qualification (terminal F_H gate); e6 in deployment; e7 in observation; e8 in retrofit.

The published `Module` is the AI inventory entry. The event stream is the audit trail. The traced call-out archive is the per-invocation forensic evidence. The F_H attestation is the committee approval. The CandidateFamily over control sets is the proportionality mechanism. The recurring `e8` is the recertification cadence.

This is implementable today on the current substrate with three small additions — risk-tier carrier on `Job`/`Module`, escalation `CandidateFamily` shape, and recurrence on `RefinementBoundary`. None require ABG runtime law changes.

Where odd_sdlc fits and where another odd project would fit better

odd_sdlc is shaped for **software delivery** worksites: request → specification → design → implementation → qualification → release → deployment → runtime return → observation → retrofit. MindForge's lifecycle is a **per-use-case AI delivery** lifecycle that overlaps but is not coextensive.

For a financial institution implementing MindForge on the GTL/ABG substrate, three deployment shapes are plausible:

1. **odd_sdlc directly** — works for FIs whose AI use case lifecycle is a software delivery lifecycle (the DBS CodeBuddy case is exactly this). odd_sdlc's existing stages map well; MindForge concerns become hooks and policy overlays on the existing graph functions.

2. **A new `odd_aigovernance` project** — a sibling of odd_sdlc whose lifecycle is the MindForge use-case lifecycle, with its own GTL Module, its own stage names matching MindForge Section 3 verbs, and its own role taxonomy aligned to MindForge's intended audience (Executives, Builders, Custodians, Use Case Owners, Business Users). This is the cleaner architectural answer for an FI building governance from the ground up.

3. **odd_sdlc + a MindForge overlay product** — odd_sdlc carries the software lifecycle; an installed `mindforge-overlay` product provides the AI-specific policy hooks, AI inventory projection, risk-tier derivation rules, and reference KRI projections. This is the lowest-cost path for an FI already using odd_sdlc to deliver AI software.

2. is the right answer if the operator's primary outcome is governed AI use across the enterprise (not just AI software delivery). (3) is the right answer if the operator's primary outcome is AI software delivery with MindForge compliance as a constraint. (1) is the right answer for proof-of-concept work and small-scale deployments.

A future ticket could carve this decision explicitly. Worth pinning before any FI starts implementation work on the substrate.

Recommended next steps

1. **Carve a feature ticket for `RiskMaterialityTier` carrier** — small, additive, unblocks Consideration 5 mapping. STDO scope: `change_class: design_reframe`, `re_entry_point: design`. ABG side: type addition; odd_sdlc side: derivation hook.
2. **Author an `odd_aigovernance` charter** under the same `specification_methodology/` companion umbrella — start from MindForge Section 3 verbs, declare role taxonomy, reuse ODD_METHOD constitutional law, scope down to AI-use-case lifecycle.
3. **Publish reference KRI projections** in odd_sdlc — replay-derived projections for hallucination rate, drift, fairness metrics, third-party model change events. These become the "KRIs" MindForge Consideration 3 requires.
4. **Add MindForge appendix to LLM_GTL_APP_BUILDER_GUIDE.md** — a one-page mapping table from MindForge Considerations to substrate primitives. Helps any LLM-driven implementation work choose the right carrier on first attempt.
5. **Write a fifth Implementation Example for the consortium** — under MAS's "ecosystem development" mandate (Executive Handbook conclusion), a worked GTL/ABG + odd_sdlc realisation of MindForge would be a concrete contribution to the toolkit and would surface any further substrate gaps not visible from this analysis alone.

Closing observation

MindForge and GTL/ABG converge on the same structural answer: AI governance is event-sourced, event-replayable, role-bound, and gate-enforced rather than process-document-described. MindForge specifies that answer in the supervisory vocabulary of

financial regulation; GTL/ABG specifies it in the language of constraint-based runtime. The mapping is closer than incidental. An FI implementing MindForge on GTL/ABG is implementing the same governance shape twice — once in policy text and once in event-sourced runtime — with the runtime side carrying the proof.

The MAS Consultation Paper on AI Risk Management (Nov 2025, finalising 2026) will move MindForge from industry handbook to supervisory expectation. At that point, an FI's MindForge compliance becomes an audited artefact. A GTL/ABG realisation produces the audit artefact as a property of operation, not as a separate document — which is exactly the structural argument in `THE_GENESIS_VISION.md` §3.

Worth pursuing.